



FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

INFORME DE PROYECTO FINAL

BASE DE DATOS AVANZADAS Y BIG DATA

Solución de Inteligencia de Negocios y Solución Big Data

Factores asociados a la no-llegada de huéspedes en el sistema Estadía de Tunki Lodge

DATOS DEL EQUIPO	INFORMACIÓN	
Autores	Ayala Hernandez, Gerardo	100%
	Bringas Veramendi, Julio Cesar Ricardo	100%
	Osorio Carbajal, Mijai	100%
	Racchumi Julon, Yeremi Adrian	100%
	Valencia Amancio, Alex	100%
Docente	Victor Isaac Chavez Bardales	
Curso / NRC	Base de Datos Avanzadas y Big Data / 226434.26055	
Repositorio	https://github.com/AlexAmancio/business-intelligence — rama main	
Período	2026-2	

Lima - Perú

2026

ORIENTACIONES PARA EL PROYECTO Y EL INFORME

Propósito del proyecto: desarrollar, integrar y evaluar una solución de Inteligencia de Negocios y una solución Big Data que respondan a una necesidad empresarial verificable. El informe debe demostrar decisiones de ingeniería, experimentación reproducible y dominio técnico de ambas soluciones.

Alineación referencial con ABET: este formato operacionaliza el Student Outcome 6 de ABET EAC mediante evidencias directas de diseño y ejecución de experimentos, análisis e interpretación de datos y conclusiones sustentadas con juicio de ingeniería.

Entregables obligatorios

N.º	ENTREGABLE	EVIDENCIA MÍNIMA
1	Informe técnico	Documento completo con citas válidas, tablas, figuras y trazabilidad de evidencias.
2	Solución BI funcional	Modelo dimensional, ETL/ELT, almacén de datos o data mart, indicadores, análisis OLAP y dashboard.
3	Solución Big Data funcional	Ingesta, almacenamiento y procesamiento escalable o distribuido, consultas/análisis y demostración técnica.
4	Paquete experimental	Dataset original y procesado, scripts/notebook, configuraciones, logs, métricas, resultados y visualizaciones.
5	Repositorio versionado	Código, documentación, historial de cambios, etiqueta de versión final y evidencia de aportes individuales.
6	Defensa y mejora	Demostración funcional, respuesta individual, retroalimentación recibida y mejoras implementadas.
7	Uso responsable de IA	Declaración de herramientas, propósito, verificación, autoría intelectual, privacidad y limitaciones.

Convenciones de presentación de evidencias

1. Identifique cada evidencia con un código único (por ejemplo, E-01, E-02) y úselo en el informe, anexos y repositorio.
2. Todo resultado debe indicar dataset, versión del código, configuración, fecha de ejecución y responsable.
3. Los enlaces deben ser accesibles para el docente y conservar una versión congelada del entregable final.
4. No reemplace el análisis técnico por capturas: las capturas complementan, pero no sustituyen, datasets, scripts, métricas, tablas y logs.
5. La evaluación del resultado de experimentación requiere evidencia individual además del producto grupal.

ÍNDICE

Actualice el índice automático después de completar el informe. Incluya además los índices de tablas, figuras y anexos.

Haga clic derecho y seleccione Actualizar campo para generar el índice.

RESUMEN

La concurrencia de clientes en Tunki Lodge viene bajando y el sistema Estadía no ofrece ninguna forma de explicar por qué. En la base operativa real (alquiler), el historial cubre solo tres meses y muestra un 14.3% de estadías terminadas en cancelación o no-show, insuficiente para medir una tendencia. El objetivo del proyecto fue diseñar una solución de Inteligencia de Negocios y una solución Big Data que, en conjunto, expliquen esa caída de concurrencia a lo largo del tiempo. Para la solución BI se construyó un datamart en esquema estrella (hotel_dw), centrado en una dimensión de tiempo, mediante un proceso ETL en SQL sobre la base operativa. Como la base real cubre muy poco historial, la solución Big Data generó un dataset sintético de 3,000,000 de filas a lo largo de tres años (2024-2026), calibrado con las proporciones reales, y lo procesó con PySpark en modo local sobre 6 núcleos. El diseño experimental comparó tres escenarios: el agregado SQL sobre datos reales, el mismo agregado en Spark sobre datos sintéticos, y un benchmark de escalabilidad variando el número de particiones de Spark (1, 4, 8 y 6). Los resultados muestran que la concurrencia mensual cae de 116,322 a 53,982 estadías (-53.6%) del primer al último mes del periodo, mientras la tasa de no-llegada sube de 21.5% a 39.2% en el mismo lapso; además, la categoría Deluxe y los viernes/lunes concentran la mayor no-llegada, y repartir el trabajo en 6 particiones redujo el tiempo del agregado de 1.22 a 0.90 segundos sobre las 3,000,000 de filas. Se concluye que la caída de concurrencia y el aumento de no-llegada se mueven juntos, no son eventos aleatorios, y que el procesamiento distribuido permite sostener ese análisis a la escala que el hotel alcanzaría en unos años.

Palabras clave: inteligencia de negocios, big data, concurrencia de clientes, modelo dimensional, Apache Spark

ABSTRACT

Customer concurrency at Tunki Lodge has been declining, and the Estadía system offers no way to explain why. In the real operational database (alquiler), the history covers only three months and shows 14.3% of stays ending in cancellation or no-show, not enough to measure a trend. The goal of this project was to design a Business Intelligence solution and a Big Data solution that together explain that decline in concurrency over time. For the BI solution, a star-schema data mart (hotel_dw), centered on a time dimension, was built through a SQL ETL process over the operational database. Since the real database covers too little history, the Big Data solution generated a synthetic dataset of 3,000,000 rows spanning three years (2024-2026), calibrated with the real proportions, and processed it with PySpark in local mode across 6 cores. The experimental design compared three scenarios: the SQL aggregate over real data, the same aggregate in Spark over synthetic data, and a scalability benchmark varying the number of Spark partitions (1, 4, 8 and 6). Results show that monthly concurrency falls from 116,322 to 53,982 stays (-53.6%) from the first to the last month of the period, while the no-show rate

rises from 21.5% to 39.2% over the same span; the Deluxe category and Fridays/Mondays also concentrate the highest no-show rates, and spreading the workload across 6 partitions reduced aggregate runtime from 1.22 to 0.90 seconds over the 3,000,000 rows. We conclude that the drop in concurrency and the rise in no-shows move together, are not random events, and that distributed processing can sustain this analysis at the scale the hotel would reach within a few years.

Keywords: business intelligence, big data, customer concurrency, dimensional model, Apache Spark

CAPÍTULO I: CONTEXTO, PROBLEMA Y OBJETIVOS

1.1 Contexto y necesidad empresarial

Tunki Lodge opera con Estadía, un sistema web propio (Spring Boot, MySQL) que ya cubre todo el ciclo de una estadía: registro de clientes, gestión de habitaciones y categorías, reservas simples y multi-habitación, check-in, consumos adicionales, check-out, cancelación, no-show y extensión de estadía, con roles de Usuario Normal y Administrador. El sistema está en producción interna y es la fuente operativa (alquiler) usada en este proyecto. La señal que motiva el proyecto es que la concurrencia de clientes al hotel viene bajando, y el equipo administrativo no tiene forma de explicar por qué. En la base operativa real, el 14.3% de las 21 estadías registradas terminó en cancelación o no-show, pero ese historial cubre apenas tres meses (agosto a octubre de 2026), insuficiente para ver una tendencia. Por eso el proyecto generó un dataset a escala, calibrado con las proporciones reales, que cubre un periodo de tres años (2024-2026): sobre ese dataset, la concurrencia mensual cae de 116,322 a 53,982 estadías (-53.6%) del primer al último mes, mientras que la tasa de no-llegada sube de 21.5% a 39.2% en el mismo periodo. Esa es la línea base cuantitativa del problema: la caída de concurrencia y el aumento de no-llegada se mueven juntos.

1.2 Formulación del problema de ingeniería

El problema de ingeniería de este proyecto es por qué ha bajado la concurrencia de clientes en Tunki Lodge. No es un problema de una sola causa: la no-llegada de un huésped (no-show o cancelación) es el evento medible más directo, pero la pregunta de fondo es si esa no-llegada se está volviendo más frecuente con el tiempo, y en qué segmento del negocio (categoría de habitación, día de la semana, anticipación de la reserva) se concentra ese deterioro. Las causas posibles existen como datos dispersos en el sistema operativo, pero nunca se consolidan a lo largo del tiempo ni se comparan periodo contra periodo. La consecuencia directa es una habitación bloqueada sin generar ingreso y una planificación de personal e insumos basada en una demanda que ya no es la de antes. Una restricción central es que el historial real del sistema (21 registros, tres meses) es demasiado corto para medir una tendencia por sí solo. El problema requiere una solución de Inteligencia de Negocios porque hace falta un modelo dimensional centrado en el tiempo (dim_tiempo) que permita comparar la concurrencia mes contra mes; y requiere una solución Big Data porque, para observar esa tendencia con suficiente historial, hace falta procesar un volumen de datos que el sistema real todavía no ha acumulado, y ese procesamiento debe demostrar que escala.

1.3 Objetivos

Objetivo general: Diseñar e implementar una solución de Inteligencia de Negocios y una solución Big Data que permitan explicar la caída de la concurrencia de clientes en el sistema Estadía de Tunki Lodge a lo largo del tiempo, identificando en qué segmentos (categoría de habitación, día de la semana, anticipación de la reserva) se concentra el aumento de no-show y cancelación, con evidencia de al menos 15 puntos porcentuales de diferencia entre los segmentos identificados y una medición explícita de la variación de concurrencia mes contra mes.

Objetivos específicos:

(BI) Construir un modelo dimensional en esquema estrella y un proceso ETL que integren clientes, habitaciones y registros de estadía del sistema Estadía en un datamart analítico

(hotel_dw).

(Big Data) Generar y procesar con Apache Spark un dataset a escala de millones de registros, calibrado con las proporciones reales, para medir el comportamiento de no-llegada bajo un volumen realista de un hotel en crecimiento.

(Experimentación) Diseñar y ejecutar al menos tres escenarios experimentales que comparen ambas soluciones, incluyendo un escenario de escalabilidad sobre el número de particiones de procesamiento.

1.4 Alcance, restricciones y supuestos

Incluye las estadías registradas en el sistema Estadía: clientes, habitaciones, categorías, consumos y los cuatro estados de un registro (ACTIVA, FINALIZADA, CANCELADA, NO_SHOW). No incluye predicción en tiempo real dentro de la aplicación web, integración con canales de reserva externos (OTA) ni datos de otros hoteles. Restricción principal: el dataset operativo real es pequeño (21 registros a la fecha del proyecto), por lo que la solución Big Data usa un dataset sintético calibrado con las proporciones reales para sostener el volumen necesario en el análisis a escala. El proyecto no cuenta, en esta etapa, con presupuesto de infraestructura cloud: el procesamiento distribuido corre en modo local[*] de Spark sobre la máquina de desarrollo del equipo. Los datos personales de clientes reales no se publican en el repositorio; solo se usan localmente para construir el datamart. Se asume que las proporciones de no-show y cancelación calibradas sobre los 21 registros reales son representativas para efectos de este análisis, aunque una muestra de ese tamaño no permite una generalización estadística estricta.

1.5 Stakeholders y criterios de éxito

STAKEHOLDER	NECESIDAD / DECISIÓN	CRITERIO DE ÉXITO	EVIDENCIA
Administrador del hotel	Anticipar qué estadías tienen mayor riesgo de no llegar, para ajustar overbooking y personal	Reducir en al menos 10 puntos porcentuales la brecha de no-llegada entre el segmento de mayor y menor riesgo	E-07
Recepción (Usuario Normal)	Contar con un tablero que muestre la ocupación y el riesgo de no-llegada del día	Tablero OLAP con al menos 3 indicadores de no-llegada actualizados	E-07
Equipo de datos	Procesar el volumen creciente de reservas sin degradar el tiempo de respuesta del análisis	Reducir el tiempo del agregado distribuido en al menos 30% al pasar de 1 a 6 particiones de Spark	E-08

1.6 Fuentes, gobierno y ética de los datos

La fuente operativa real es la base de datos alquiler del sistema Estadía (MySQL), de propiedad del equipo, de uso interno. Los datos personales de clientes (nombres, identificación, contacto) se tratan solo localmente y no se publican en el repositorio versionado. El dataset a escala usado en la solución Big Data es 100% sintético: se genera a partir de distribuciones estadísticas calibradas con las proporciones reales observadas, sin contener ningún dato personal real. La calidad del ETL se valida por reconciliación de conteos entre el origen y el datamart (sección 6.1). Los datasets y resultados se conservan versionados en el repositorio. El principal riesgo de sesgo

es que la muestra real (21 registros) es demasiado pequeña para generalizar por sí sola; por eso se declara explícitamente que los hallazgos cuantitativos de la solución Big Data provienen del dataset sintético calibrado, y no deben leerse como una medición exacta del comportamiento real del hotel.

CAPÍTULO II: ANTECEDENTES Y FUNDAMENTOS

2.1 Antecedentes

Antecedente 1. Antonio, N., de Almeida, A., y Nunes, L. (2017). Predicting hotel bookings cancellation with a machine learning classification model. En 2017 16th IEEE International Conference on Machine Learning and Applications (ICMLA) (pp. 1049-1054). IEEE.

Problema: los hoteles no distinguen, en el momento de la reserva, cuáles probablemente se cancelarán, lo que distorsiona su cálculo de demanda neta. Método: un modelo de clasificación entrenado a diario sobre los datos del sistema de gestión de propiedades (PMS) del hotel. Datos: reservas reales de una cadena hotelera portuguesa. Métricas: precisión del modelo para recalcular la demanda neta. Resultado: el modelo permite descontar cancelaciones probables de la demanda bruta. Aporte al proyecto: motivó incluir la anticipación de la reserva como variable del dataset sintético usado en la solución Big Data de este informe.

Antecedente 2. Antonio, N., de Almeida, A., y Nunes, L. (2019). Big data in hotel revenue management: Exploring cancellation drivers to gain insights into booking cancellation behavior. *Cornell Hospitality Quarterly*, 60(4), 298-319. <https://doi.org/10.1177/1938965519851466>

Problema: los hoteles generan grandes volúmenes de datos de reserva sin aprovecharlos para entender la cancelación. Método: análisis de factores asociados a la cancelación ("cancellation drivers") sobre datos históricos a gran escala. Datos: reservas de varios hoteles portugueses. Métricas: variables asociadas a la cancelación (antelación, segmento, temporada). Resultado: la antelación de la reserva y el tipo de habitación están entre los factores más asociados a la cancelación. Aporte al proyecto: respalda directamente segmentar el análisis por categoría de habitación y por anticipación, como se hizo en el datamart y en el procesamiento Spark de este proyecto.

Antecedente 3. Chen, S., Ngai, E. W. T., Ku, Y., Xu, Z., Gou, X., y Zhang, C. (2023). Prediction of hotel booking cancellations: Integration of machine learning and probability model based on interpretable feature interaction. *Decision Support Systems*, 170, 113959. <https://doi.org/10.1016/j.dss.2023.113959>

Problema: los modelos de predicción de cancelación suelen ser precisos pero poco interpretables para la operación del hotel. Método: integración de un modelo probabilístico (red bayesiana) con un modelo de aprendizaje automático interpretable. Datos: reservas hoteleras reales. Métricas: desempeño predictivo y ranking de variables por importancia. Resultado: el modelo integrado mejora la interpretabilidad sin perder desempeño. Aporte al proyecto: valida que reportar la tasa de no-llegada segmentada (por categoría, día y anticipación, como en el capítulo VI de este informe) es más útil para la decisión operativa que un único indicador agregado.

Antecedente 4. Kirmani, M. M. (2017). Dimensional modeling using star schema for data warehouse creation. *Oriental Journal of Computer Science and Technology*, 10(3), 745-754.

Problema: falta de un procedimiento claro para transformar un modelo entidad-relación operativo en un modelo dimensional. Método: un algoritmo de transformación de diagramas entidad-relación a esquema estrella. Datos: casos de estudio de bases de datos transaccionales. Métricas: cobertura de tablas de hechos y dimensiones generadas. Resultado: un procedimiento reproducible para construir el esquema estrella a partir del modelo operativo. Aporte al proyecto: se siguió este procedimiento para transformar el modelo entidad-relación de alquiler (registro, cliente, habitacion, categoria) en el esquema estrella hotel_dw de este proyecto.

Antecedente 5. Golfarelli, M., y Rizzi, S. (2018). From star schemas to big data: 20+ years of data warehouse research. En S. Flesca, S. Greco, E. Masciari, y D. Saccà (Eds.), *A comprehensive guide through the Italian*

database research over the last 25 years (Studies in Big Data, Vol. 31, pp. 93-107). Springer. https://doi.org/10.1007/978-3-319-61893-7_6

Problema: cómo evoluciona el data warehouse clásico frente a la llegada del Big Data. Método: revisión de más de 20 años de investigación en almacenamiento de datos. Datos: literatura académica del área (no aplica dataset). Métricas: no aplica, es una revisión. Resultado: los principios del modelado dimensional siguen vigentes como capa de consumo, incluso cuando el procesamiento subyacente es distribuido. Aporte al proyecto: sustenta la arquitectura integrada de este informe, donde Spark procesa el volumen sintético y el esquema estrella hotel_dw sigue siendo la capa de consumo para el análisis OLAP.

2.2 Fundamentos de Inteligencia de Negocios

La integración de datos consiste en reunir en un único repositorio analítico la información que en el sistema Estadía vive repartida en tablas operativas independientes (cliente, habitacion, categoria, registro, consumo). El mecanismo usado para esa integración es un proceso ETL: se extraen (Extract) las filas de alquiler, se transforman (Transform) —por ejemplo, calculando el saldo pendiente como tarifa más consumos menos lo pagado, y derivando las banderas es_no_show, es_cancelada y es_finalizada a partir del estado del registro— y se cargan (Load) en un almacén separado, hotel_dw. El modelado dimensional (Kirmani, 2017) organiza ese almacén en una tabla de hechos, fact_estadía, con una fila por estadía y sus medidas (noches, tarifa, saldo pendiente), rodeada de tablas de dimensiones (dim_cliente, dim_habitacion, dim_categoria, dim_tiempo, dim_usuario) que describen el contexto de cada hecho. Como hotel_dw cubre un único proceso de negocio —la estadía— y no toda la operación del hotel, técnicamente es un data mart y no un data warehouse corporativo completo. Sobre ese modelo se ejecutan consultas OLAP: agregaciones multidimensionales que cruzan la tabla de hechos con una o más dimensiones, como la tasa de no-llegada por categoría de habitación o por día de la semana (capítulo III, sección 3.4.4). Esas consultas alimentan los indicadores del proyecto (tasa de no-llegada, saldo pendiente por estado, ocupación) y, en última instancia, la visualización que le da sentido a un usuario no técnico, como el administrador del hotel.

2.3 Fundamentos de Big Data

El sistema Estadía real solo tiene 21 registros de estadía, un volumen que cualquier motor relacional procesa sin dificultad. Sin embargo, un hotel en operación continua y con varios años de historial, o una cadena con varias sedes, generaría fácilmente cientos de miles o millones de registros de reserva. La justificación de Big Data en este proyecto no es la herramienta en sí, sino la necesidad de comprobar que el análisis de no-llegada se sostiene cuando el volumen crece varios órdenes de magnitud: por eso se construyó un dataset sintético de 3,000,000 de filas, calibrado con las proporciones reales, y se midió el tiempo real de un mismo agregado al variar el número de particiones de procesamiento (1, 4, 8 y 6 núcleos). La variedad es moderada —los datos siguen siendo estructurados y provienen de un modelo relacional—, pero la escalabilidad y el procesamiento distribuido sí son una exigencia medible: Zaharia et al. (2010) muestran que Spark conserva la escalabilidad horizontal y la tolerancia a fallos de MapReduce, pero permite reutilizar en memoria un mismo conjunto de datos entre operaciones sucesivas, lo que resulta más eficiente que recalcular desde disco cada vez que se necesita un nuevo agregado sobre las mismas 3,000,000 de filas.

2.4 Tecnologías, estándares y trabajos relacionados

Para la capa BI se comparó alojar hotel_dw en el mismo motor MySQL del OLTP alquiler frente a construirlo en SQL Server. Se optó por SQL Server porque ofrece un soporte más maduro de funciones OLAP (funciones de ventana, agregaciones jerárquicas) y porque el equipo ya lo usa como estándar en su infraestructura de datos,

sin que el volumen del datamart BI (decenas de miles de filas en el escenario real) exija por sí solo una infraestructura adicional de cómputo. Para la capa Big Data se comparó procesar el dataset sintético con pandas (secuencial, en un solo hilo) frente a PySpark 4.2.0 sobre Java 17 en modo local[*] (distribuido entre los núcleos disponibles de la máquina). Se eligió PySpark por tres razones: permite demostrar el modelo de procesamiento distribuido de Zaharia et al. (2010) sin requerir un clúster pagado, su API de DataFrames se mantiene legible en Python (el mismo lenguaje usado en el generador del dataset sintético), y es el estándar de facto para procesamiento distribuido en memoria sobre datos estructurados. Los criterios de selección fueron rendimiento y escalabilidad (medidos con el benchmark de particiones de la sección 3.5.4), interoperabilidad (Spark lee CSV estándar sin dependencias adicionales), seguridad (todo el procesamiento corre localmente, sin exponer datos a servicios externos) y costo (sin clúster ni licencias, solo la máquina de desarrollo del equipo).

CAPÍTULO III: DISEÑO DE INGENIERÍA DE LAS SOLUCIONES

3.1 Requerimientos funcionales y no funcionales

RF-01 El proceso ETL debe extraer clientes, habitaciones, categorías y registros desde alquiler y cargarlos en hotel_dw sin pérdida de filas.

RF-02 El datamart debe permitir calcular la tasa de no-llegada (no-show + cancelada) segmentada por categoría de habitación, día de la semana y anticipación de la reserva.

RF-03 El proceso Big Data debe generar y procesar un dataset sintético de al menos 1,000,000 de registros.

RF-04 El procesamiento distribuido debe entregar los mismos indicadores de no-llegada que el datamart BI, para que ambas soluciones sean comparables.

RNF-01 El ETL de alquiler a hotel_dw debe ejecutarse en menos de 5 segundos sobre el volumen operativo actual.

RNF-02 El agregado principal de Spark sobre el dataset sintético (3,000,000 de filas) debe completarse en menos de 5 segundos en modo local[*].

RNF-03 El repositorio debe permitir reproducir ambos procesos desde cero solo con Python, MySQL, SQL Server y Java instalados.

3.2 Alternativas y decisiones de arquitectura

ALTERNATIVA	CRITERIOS	VENTAJAS	LIMITACIONES / RIESGOS	DECISIÓN
Motor del datamart BI	Costo, integración con el OLTP existente, curva de aprendizaje del equipo	Soporte OLAP más maduro que MySQL (funciones de ventana, agregaciones jerárquicas); ya es el estándar de infraestructura de datos del equipo	Sin capacidades OLAP nativas avanzadas (cubos, particionamiento analítico)	Seleccionada: SQL Server (hotel_dw), por su soporte OLAP y por ser el estándar de infraestructura de datos del equipo
Motor de procesamiento Big Data	Escalabilidad, curva de aprendizaje del equipo, madurez del ecosistema	PySpark: procesamiento distribuido en memoria, mismo lenguaje (Python) que el resto del proyecto	Requiere JVM (Java) y más memoria que un script secuencial equivalente	Seleccionada: PySpark en modo local[*], por escalabilidad demostrable y por ser el estándar de facto para procesamiento distribuido en memoria
Formato del dataset a escala	Legibilidad, compatibilidad, tamaño en disco	CSV: legible, compatible con SQL Server, Excel y Spark sin dependencias adicionales	Mayor tamaño en disco que un formato binario columnar como Parquet	Seleccionada: CSV para esta entrega; Parquet queda como mejora futura (ver recomendaciones, capítulo VII)

3.3 Arquitectura integrada y flujo de datos

El sistema Estadía (Spring Boot y MySQL alquiler) es la única fuente OLTP del proyecto y desde ahí se abren dos rutas. La ruta BI extrae, transforma y carga los datos reales mediante un ETL en SQL hacia hotel_dw (esquema estrella en SQL Server), que se consulta con T-SQL estándar. La ruta Big Data genera un dataset sintético calibrado con las proporciones reales y lo procesa con PySpark en modo local[*], calculando los mismos indicadores de no-llegada a escala y ejecutando el benchmark de particiones. Ambas rutas convergen en el mismo conjunto de indicadores (tasa de no-llegada por categoría, por día de la semana y por anticipación de la reserva), lo que permite comparar en el capítulo VI los resultados de la ruta BI (datos reales, volumen pequeño) contra los de la ruta Big Data (datos sintéticos, volumen grande). El límite entre ambas soluciones es claro: todo lo que ocurre en las bases relacionales (alquiler en MySQL y hotel_dw en SQL Server) es la solución BI; todo lo que ocurre en PySpark sobre el archivo CSV sintético es la solución Big Data. En cuanto a seguridad, el sistema Estadía ya controla el acceso mediante Spring Security, y tanto el datamart como el dataset sintético se procesan de forma local, sin exposición a red. Los tres componentes se despliegan en la misma máquina de desarrollo del equipo, sin clúster, lo cual se declara como restricción en la sección 1.4.

Figura 1. Arquitectura integrada de la solución. Fuentes (MySQL alquiler) -> ETL -> Datamart BI (hotel_dw, SQL Server) -> consultas OLAP, y en paralelo Fuentes calibradas -> generador sintético -> PySpark local[*] -> indicadores a escala. Ambas rutas convergen en los mismos indicadores de no-llegada. Diagrama disponible en el repositorio como bi/arquitectura.png (evidencia E-05).

3.4 Diseño de la solución de Inteligencia de Negocios

3.4.1 Requisitos, usuarios e indicadores

Pregunta de negocio 1: ¿la concurrencia de clientes está bajando mes a mes, y en qué medida eso se explica por el aumento de no-llegada? Usuario: Administrador del hotel. Decisión que apoya: si conviene ajustar la capacidad, el personal o la política de overbooking del periodo siguiente. KPI 1: concurrencia mensual (conteo de estadías) y su variación porcentual respecto al primer mes del periodo. Fórmula: (estadías del mes – estadías del primer mes) / estadías del primer mes, por 100. Granularidad: por año-mes del check-in. Pregunta de negocio 2: ¿qué categoría de habitación y qué día de la semana concentran más no-llegada dentro de esa caída de concurrencia? KPI 2: porcentaje de no-llegada. Fórmula: (no_show + cancelada) / total de estadías, por 100. Granularidad: por categoría de habitación, por día de la semana y por año-mes del check-in. Frecuencia: consulta bajo demanda sobre hotel_dw (no hay actualización en tiempo real en esta entrega). Fuente: hotel_dw.fact_estadia, unida a dim_habitacion, dim_categoria y dim_tiempo. Meta: confirmar si la caída de concurrencia y el aumento de no-llegada se mueven juntos, e identificar el segmento con la mayor brecha respecto al promedio general del hotel.

3.4.2 Modelo dimensional y almacén de datos

El modelo conceptual es un esquema estrella con una tabla de hechos, fact_estadia, cuyo grano es una fila por registro de estadía. Sus medidas son noches, tarifa, monto_pagado, total_consumos, saldo_pendiente y las banderas es_no_show, es_cancelada y es_finalizada. Se relaciona con cinco dimensiones: dim_tiempo (fecha del check-in, año, mes, día de la semana, trimestre), dim_categoria (nombre y precio base de la habitación), dim_habitacion (número, piso, aforo), dim_cliente (nombres, país, ciudad, ocupación) y dim_usuario (quién registró la estadía). A nivel lógico y físico, id_registro es la clave primaria de los hechos y cada dimensión aporta su propia clave primaria como clave foránea en fact_estadia (id_cliente, id_habitacion, id_usuario,

id_tiempo_checkin); id_categoria es, a su vez, clave foránea dentro de dim_habitacion. No se implementaron dimensiones de cambio lento (SCD) porque el alcance del proyecto es una fotografía del estado actual del OLTP, no un historial de cambios. El script completo, con la definición de cada tabla y tipo de dato, está en bi/datamart_estadia.sql (evidencia E-05).

3.4.3 Procesos ETL/ELT y calidad

El script bi/datamart_estadia.sql recrea hotel_dw desde cero en SQL Server (DROP DATABASE IF EXISTS seguido de CREATE DATABASE), lo que hace el proceso idempotente y automatizable con un único comando: sqlcmd -S localhost -i bi/datamart_estadia.sql. Como alquiler y hotel_dw viven en motores distintos, la extracción se hace con un script (bi/extraer_generar_tsq.py) que consulta las tablas de alquiler (cliente, habitacion, categoria, usuario, registro) y genera las sentencias de carga en T-SQL. La transformación calcula, para cada estadía, el saldo pendiente como tarifa más el total de consumos asociados menos el monto pagado, y deriva las banderas es_no_show, es_cancelada y es_finalizada a partir del campo estado del registro. La carga inserta cada fila en su tabla de hotel_dw apoyada en las claves foráneas declaradas, para que el motor rechace cualquier fila cuyo cliente o habitación no exista, en vez de perderla en silencio. La reconciliación se verificó de forma directa: los 21 registros de alquiler.registro corresponden a exactamente 21 filas en hotel_dw.fact_estadia después de correr el script (evidencia E-06, detalle cuantitativo en la sección 6.1). Esto cumple el requisito obligatorio de evidenciar la automatización de procesos de base de datos.

3.4.4 OLAP, reportes y dashboards

El script incluye tres consultas OLAP sobre hotel_dw, cada una ligada a una decisión distinta: (1) tasa de no-llegada por categoría de habitación, que apoya dónde reforzar el seguimiento de reservas según el tipo de habitación; (2) tasa de no-llegada por día de la semana del check-in, que apoya qué días conviene reforzar la confirmación telefónica o el recordatorio al huésped; y (3) ingresos por tarifa, ingresos por consumos y saldo pendiente total agrupados por estado de la estadía, que apoya el seguimiento de cobranza. Las tres son consultas SQL ejecutables sobre hotel_dw desde cualquier cliente (fuente verificable) y quedan como base directa para un tablero (Power BI u otra herramienta de visualización) en una siguiente iteración del proyecto.

3.5 Diseño de la solución Big Data

3.5.1 Justificación y requisitos Big Data

El dataset sintético tiene 3,000,000 de filas frente a los 21 registros reales del sistema, es decir, más de 140,000 veces el volumen operativo actual, un orden de magnitud comparable al que acumularía una cadena hotelera con varios años de operación. En esta entrega no se maneja streaming (los datos se generan por lote, no en tiempo real), lo que se declara como limitación explícita; la variedad es estructurada y multi-tabla, no semi-estructurada. La condición medible que sí se evaluó es la escalabilidad del procesamiento: qué tan bien responde el mismo agregado de no-llegada cuando se reparte entre distintos números de particiones sobre un volumen que un motor relacional de fila por fila tardaría notablemente más en procesar.

3.5.2 Arquitectura y componentes

Ingesta: por lote, con spark.read.csv e inferencia automática de esquema sobre bigdata/estadias_sinteticas.csv. Almacenamiento: sistema de archivos local (no se usó HDFS, al operar en modo local[*]). Procesamiento distribuido: Spark SQL y la API de DataFrames, repartiendo el trabajo entre los 6 núcleos disponibles de la máquina de desarrollo. Orquestación: ejecución directa de un único script Python (bigdata/analisis_pyspark.py), sin orquestador externo, dado que es un solo job. Consulta: agregaciones groupBy().agg() sobre el DataFrame en memoria (df.cache()). Monitoreo: la interfaz web de Spark y los logs de consola durante la ejecución.

Seguridad: el dataset es sintético y no contiene datos personales reales, por lo que no se requirió cifrado adicional. Versiones: PySpark 4.2.0, Java 17 (OpenJDK, Homebrew) y Python 3.14.

3.5.3 Modelo y ciclo de vida de datos

El dataset se almacena en un único archivo CSV con encabezado, cuyo esquema es inferido automáticamente por Spark al leerlo (columnas categoría, fecha_checkin, día_semana, noches, num_huespedes, anticipacion_horas, tarifa, consumos, monto_pagado, estado, entre otras). No se aplicó particionamiento físico en disco en esta entrega: el archivo es uno solo, aunque Spark lo reparte en memoria en tiempo de ejecución según el número de particiones configurado. No se usó un catálogo de metadatos externo (por ejemplo, Hive Metastore), dado el alcance actual del proyecto. El generador (`bigdata/generar_dataset_sintetico.py`) asigna un `id_registro` secuencial único, por lo que no hay duplicados por construcción. El concepto de datos "tardíos" no aplica: el dataset se genera una sola vez por lote, no llega de forma continua.

3.5.4 Escalabilidad, resiliencia y operación

La escalabilidad se evalúa con un benchmark real: el mismo agregado (promedio de no-llegada por categoría y día de la semana) se ejecuta sobre las mismas 3,000,000 de filas variando el número de particiones (1, 4, 8 y 6, este último el número de núcleos disponibles), midiendo el tiempo de reloj real de cada ejecución (resultados en la sección 6.3). La tolerancia a fallos se apoya en la resiliencia nativa de los RDD de Spark, que permite recomputar datos perdidos a partir de su linaje; no se probó una inyección de fallos real en esta entrega, lo que queda declarado como limitación en la sección 4.7. El uso de recursos está acotado por la máquina de desarrollo: 6 núcleos y 4 GB de memoria asignados al driver de Spark. Como propuesta futura (no implementada) queda ejecutar el mismo pipeline sobre un clúster real con más de un nodo, para medir la escalabilidad horizontal más allá de los núcleos de una sola máquina.

3.5.5 Resultados

La ejecución de `bigdata/analisis_pyspark.py` sobre las 3,000,000 de filas produce, por consola y en el archivo `bigdata/resultados_pyspark.json` (evidencia E-08), cuatro tablas —por categoría de habitación, por día de la semana, por franja de anticipación de la reserva y por mes (esta última es la tendencia de concurrencia: 116,322 a 53,982 estadías, -53.6%, con la tasa de no-llegada subiendo de 21.5% a 39.2%)— más el benchmark de particiones. El detalle cuantitativo de estos resultados y su interpretación se presenta en el capítulo VI de este informe, una vez que se compara contra los resultados de la solución BI sobre datos reales.

3.6 Seguridad, privacidad y riesgos

La principal amenaza es la exposición de datos personales reales de los clientes del hotel; se mitiga no publicando la base alquiler ni sus datos en el repositorio, y usando exclusivamente el dataset sintético para la solución Big Data. El acceso a la aplicación web ya está controlado por Spring Security (roles Usuario Normal y Administrador, con bloqueo temporal por intentos fallidos de login), y tanto MySQL y SQL Server como el procesamiento de Spark corren en la máquina local del equipo, sin exposición a internet. El MySQL de desarrollo no tiene contraseña configurada, lo que se declara como un riesgo aceptado únicamente en el entorno local de desarrollo de este proyecto, nunca en un entorno productivo. Como respaldo y auditoría, los scripts SQL son reproducibles y recrean el datamart completo desde cero (`DROP DATABASE IF EXISTS` seguido de `CREATE DATABASE`), lo que funciona como mecanismo de recuperación del propio proceso. En privacidad, el dataset a escala es enteramente sintético. En sesgo, se declara explícitamente que ese dataset fue calibrado con solo 21 observaciones reales, por lo que sus proporciones exactas no deben interpretarse como una medición precisa del comportamiento del hotel real, sino como un ejercicio de escala con señal realista (ampliado en las secciones 4.7 y 8.1).

3.7 Plan de despliegue y aceptación

Entorno: desarrollo local (la máquina del equipo) para ambas soluciones; no se definió un entorno de producción en esta entrega. Dependencias: MySQL 8 (OLTP alquiler), SQL Server 2022 (datamart hotel_dw), Python 3.14, PySpark 4.2.0 y Java 17. Pasos de despliegue: (1) restaurar o mantener la base alquiler del sistema Estadía en ejecución; (2) correr bi/datamart_estadia.sql para reconstruir hotel_dw; (3) generar el dataset sintético con bigdata/generar_dataset_sintetico.py; (4) ejecutar bigdata/analisis_pyspark.py. Rollback: al ser hotel_dw un proceso idempotente (DROP DATABASE IF EXISTS seguido de CREATE DATABASE), basta con volver a correr el script para revertir cualquier cambio manual sobre el datamart. Responsables: el equipo de desarrollo del proyecto. Criterios de aceptación: el ETL reconcilia el 100% de las filas de alquiler.registro contra hotel_dw.fact_estadia (sección 6.1) y el job de Spark procesa las 3,000,000 de filas sintéticas sin error, generando bigdata/resultados_pyspark.json. La demostración funcional consiste en ejecutar ambos scripts en vivo.

CAPÍTULO IV: DISEÑO DEL EXPERIMENTO

Evidencia principal de la rúbrica 6.1: el capítulo debe permitir que otra persona reproduzca el experimento y comprenda por qué evalúa apropiadamente las soluciones BI y Big Data.

4.1 Pregunta experimental, objetivo e hipótesis

Formule una pregunta medible. Declare objetivo, hipótesis o resultado esperado y relación con los objetivos del proyecto. Si no corresponde hipótesis, justifique el enfoque comparativo o descriptivo.

4.2 Escenario y datasets

DATASET	FUENTE Y VERSIÓN	VOLUMEN / FORMATO	CALIDAD Y PRIVACIDAD	USO EN BI / BIG DATA	EVIDENCIA
Original	<<Origen, fecha, licencia>>	<<Filas, GB, formatos>>	<<Perfilado, restricciones>>	<<Uso>>	E-01
Procesado	<<Versión / hash>>	<<Filas, GB, particiones>>	<<Reglas y resultado>>	<<Uso>>	E-02
Prueba / sintético	<<Método de generación>>	<<Escalas>>	<<Validez y límites>>	<<Uso>>	E-03

4.3 Variables, métricas y criterios de aceptación

TIPO	VARIABLE / MÉTRICA	DEFINICIÓN Y UNIDAD	MÉTODO DE MEDICIÓN	META / UMBRAL
Independiente	<<Volumen/ configuración/consulta>>	<<Definición>>	<<Cómo se controla>>	<<N/A o niveles>>
Dependiente	<<Latencia/throughput/ calidad>>	<<Unidad y fórmula>>	<<Instrumento/log>>	<<Meta>>
Control	<<Hardware, concurrencia, caché...>>	<<Valor fijo>>	<<Verificación>>	<<Tolerancia>>
Negocio	<<KPI o precisión de información>>	<<Unidad y fórmula>>	<<Fuente>>	<<Meta>>

4.4 Entorno, herramientas y reproducibilidad

Registre hardware, sistema operativo, motor/servicios, versiones, configuración, recursos, dependencias, semilla aleatoria si aplica, fecha y responsable. Enlace scripts de instalación o archivo de entorno.

COMPONENTE	VERSIÓN / CONFIGURACIÓN	PROPÓSITO	EVIDENCIA
<<Componente>>	<<Versión y parámetros>>	<<Función>>	<<Archivo/ enlace>>

COMPONENTE	VERSIÓN / CONFIGURACIÓN	PROPÓSITO	EVIDENCIA
<<Componente>>	<<Versión y parámetros>>	<<Función>>	<<Archivo/ enlace>>
<<Componente>>	<<Versión y parámetros>>	<<Función>>	<<Archivo/ enlace>>
<<Componente>>	<<Versión y parámetros>>	<<Función>>	<<Archivo/ enlace>>

4.5 Matriz de escenarios experimentales

Incluya como mínimo tres escenarios o ejecuciones comparables que cubran ambas soluciones. La evaluación de Big Data debe incluir al menos un escenario de rendimiento o escalabilidad. Repita mediciones cuando corresponda y reporte variabilidad.

ID	SOLUCIÓN	CAMBIO / CONFIGURACIÓN	DATOS	MÉTRICAS	CONTROLES	RESULTADO ESPERADO
EXP-01	BI	<<Consulta/modelo/ configuración>>	<<Dataset/ escala>>	<<Métricas>>	<<Constantes> >	<<Resultado>>
EXP-02	Big Data	<<Particiones/nodos/ proceso>>	<<Dataset/ escala>>	<<Métricas>>	<<Constantes> >	<<Resultado>>
EXP-03	Comparativo / integrado	<<Escenario>>	<<Dataset/ escala>>	<<Métricas>>	<<Constantes> >	<<Resultado>>

4.6 Procedimiento reproducible

1. Preparar y validar el entorno a partir de una versión identificada del repositorio.
2. Obtener el dataset original y verificar integridad, licencia y perfil de calidad.
3. Ejecutar la preparación/transformación y registrar el dataset procesado y sus controles.
4. Ejecutar cada escenario con la misma política de calentamiento, caché, concurrencia y número de repeticiones.
5. Capturar automáticamente logs, tiempos, uso de recursos, errores y resultados de negocio.
6. Consolidar resultados, calcular medidas resumen y variabilidad, generar tablas y visualizaciones.
7. Contrastar resultados con los criterios de aceptación y registrar desviaciones o incidentes.

4.7 Riesgos, validez y limitaciones del experimento

Identifique amenazas a la validez, sesgos del dataset, restricciones del entorno, incertidumbre, repetibilidad, límites de generalización y acciones de mitigación.

CAPÍTULO V: IMPLEMENTACIÓN Y CONTROL DE EVIDENCIAS

5.1 Estructura y versión del repositorio

Incluya URL, rama/etiqueta final, hash de commit, fecha, estructura de carpetas, instrucciones de ejecución, licencias y dependencias. El README debe permitir reproducir ambas soluciones y el experimento.

5.2 Implementación de la solución BI

Documente los componentes implementados, decisiones, fragmentos relevantes, pruebas y evidencias. Enlace scripts ETL/ELT, DDL, modelo dimensional, consultas, reportes y dashboard funcional.

5.3 Implementación de la solución Big Data

Documente ingesta, almacenamiento, procesamiento, particionamiento/distribución, consultas, orquestación, monitoreo y evidencias. Incluya comandos, scripts o notebooks ejecutables y archivos de configuración.

5.4 Calidad, seguridad y pruebas

Presente pruebas unitarias, integración, calidad de datos, seguridad y aceptación. Relacione cada prueba con un requisito, resultado y evidencia.

ID PRUEBA	REQUISITO	ENTRADA / CONDICIÓN	RESULTADO ESPERADO	RESULTADO	EVIDENCIA
T-0X	RF/RNF-0X	<<Entrada>>	<<Esperado>>	<<Obtenido / estado>>	<<Enlace>>
T-0X	RF/RNF-0X	<<Entrada>>	<<Esperado>>	<<Obtenido / estado>>	<<Enlace>>
T-0X	RF/RNF-0X	<<Entrada>>	<<Esperado>>	<<Obtenido / estado>>	<<Enlace>>
T-0X	RF/RNF-0X	<<Entrada>>	<<Esperado>>	<<Obtenido / estado>>	<<Enlace>>

5.5 Registro de retroalimentación y mejoras

FECHA	RETROALIMENTACIÓN	ACCIÓN / DECISIÓN	EVIDENCIA ANTES-DESPUÉS	RESP.
<<Fecha>>	<<Observación docente/usuario>>	<<Cambio o justificación>>	E-12 / <<enlace>>	<<Integrante >>
<<Fecha>>	<<Observación docente/usuario>>	<<Cambio o justificación>>	E-12 / <<enlace>>	<<Integrante >>
<<Fecha>>	<<Observación docente/usuario>>	<<Cambio o justificación>>	E-12 / <<enlace>>	<<Integrante >>

5.6 Contribuciones individuales

INTEGRANTE	RESPONSABILIDADES	COMMITTS / ARTEFACTOS	VALIDACIÓN DEL EQUIPO	%
<<Nombre>>	<<Aportes verificables>>	<<Enlaces/commits>>	<<Conformidad>>	<<%>>
<<Nombre>>	<<Aportes verificables>>	<<Enlaces/commits>>	<<Conformidad>>	<<%>>
<<Nombre>>	<<Aportes verificables>>	<<Enlaces/commits>>	<<Conformidad>>	<<%>>
<<Nombre>>	<<Aportes verificables>>	<<Enlaces/commits>>	<<Conformidad>>	<<%>>

5.7 Declaración de uso responsable de inteligencia artificial

HERRAMIENTA	FINALIDAD	CONTENIDO UTILIZADO	VERIFICACIÓN HUMANA	RIESGOS / PRIVACIDAD
<<Herramienta o 'No se utilizó'>>	<<Tarea>>	<<Código/texto/datos>>	<<Cómo se verificó>>	<<Controles y limitaciones>>
<<Herramienta o 'No se utilizó'>>	<<Tarea>>	<<Código/texto/datos>>	<<Cómo se verificó>>	<<Controles y limitaciones>>

CAPÍTULO VI: EJECUCIÓN, RESULTADOS Y ANÁLISIS DE DATOS

Evidencia principal de la rúbrica 6.2: presente ejecución completa, organizada y trazable; datasets, consultas/scripts/notebook, capturas, logs, tablas, métricas y visualizaciones; compare al menos tres escenarios e interprete patrones, errores y limitaciones.

6.1 Validación del dataset original y procesado

CONTROL	ORIGINAL	PROCESADO	REGLA / UMBRAL	RESULTADO Y EVIDENCIA
Registros / tamaño	<<Valor>>	<<Valor>>	<<Reconciliación>>	<<Resultado / E-06>>
Compleitud	<<%>>	<<%>>	<<Meta>>	<<Resultado>>
Duplicados	<<N / %>>	<<N / %>>	<<Meta>>	<<Resultado>>
Consistencia / validez	<<%>>	<<%>>	<<Meta>>	<<Resultado>>

6.2 Registro de ejecuciones

EJECUCIÓN	VERSIÓN / FECHA	ESCENARIO	CONFIGURACIÓN	ESTADO / LOG
RUN-0X	<<Commit y fecha>>	EXP-0X	<<Parámetros>>	<<Exitoso/error y enlace>>
RUN-0X	<<Commit y fecha>>	EXP-0X	<<Parámetros>>	<<Exitoso/error y enlace>>
RUN-0X	<<Commit y fecha>>	EXP-0X	<<Parámetros>>	<<Exitoso/error y enlace>>
RUN-0X	<<Commit y fecha>>	EXP-0X	<<Parámetros>>	<<Exitoso/error y enlace>>
RUN-0X	<<Commit y fecha>>	EXP-0X	<<Parámetros>>	<<Exitoso/error y enlace>>

6.3 Tabla consolidada de resultados

Reporte unidades, número de repeticiones y medida de variabilidad. No seleccione únicamente la mejor ejecución; explique exclusiones y valores atípicos.

ESCENARIO	SOLUCIÓN	ESCALA / CONFIG.	LATENCIA	THROUGHPUT	RECURSOS / CALIDAD	EVID.
EXP-01	BI	<<Valor>>	<<ms/s>>	<<qps/filas-s>>	<<CPU/RAM/% calidad>>	E-07
EXP-02	Big Data	<<Valor>>	<<ms/s>>	<<MB-s/eventos-s>>	<<CPU/RAM/nodos>>	E-08

ESCENARIO	SOLUCIÓN	ESCALA / CONFIG.	LATENCIA	THROUGHPUT	RECURSOS / CALIDAD	EVID.
EXP-03	Comparativo	<<Valor>>	<<ms/s>>	<<Unidad>>	<<Valor>>	E-09

6.4 Visualizaciones y dashboards

Inserte visualizaciones legibles con título, ejes, unidades, leyenda, fuente y código de evidencia. Explique qué decisión permite tomar cada una. Incluya capturas del dashboard BI y salidas de la solución Big Data, con enlaces funcionales cuando corresponda.

Figura X. <<Título de la visualización>>. Fuente: elaboración propia a partir de <<dataset/ejecución>>.

6.5 Análisis e interpretación técnica

1. Contraste cada resultado con el objetivo, la hipótesis o el resultado esperado.
2. Compare los escenarios y explique diferencias de rendimiento, escalabilidad, calidad o valor para el negocio.
3. Identifique patrones, errores, cuellos de botella, anomalías y limitaciones con evidencia cuantitativa.
4. Explique los compromisos de ingeniería entre precisión, latencia, costo, complejidad, seguridad y mantenibilidad.
5. Diferencie correlación, causalidad e inferencias que el experimento no permite sostener.

6.6 Verificación de funcionalidad y completitud

PRODUCTO	CRITERIOS DE ACEPTACIÓN	RESULTADO	ESTADO	EVIDENCIA
Solución BI	<<Criterios funcionales y no funcionales>>	<<Resultado>>	<<Cumple / parcial / no cumple>>	E-13
Solución Big Data	<<Criterios funcionales y no funcionales>>	<<Resultado>>	<<Cumple / parcial / no cumple>>	E-14
Integración / despliegue	<<Criterios>>	<<Resultado>>	<<Estado>>	<<Evidencia>>

CAPÍTULO VII: CONCLUSIONES TÉCNICAS Y RECOMENDACIONES

Evidencia principal de la rúbrica 6.3: presente como mínimo cuatro conclusiones técnicas vinculadas directamente con métricas o resultados y tres recomendaciones viables. Justifique decisiones con evidencia y juicio de ingeniería.

7.1 Matriz objetivo-resultado-conclusión-recomendación

OBJETIVO	RESULTADO / MÉTRICA	CONCLUSIÓN TÉCNICA	RECOMENDACIÓN / DECISIÓN	EVIDENCIA
OE-0X	<<Valor y escenario>>	<<Conclusión sustentada>>	<<Acción viable>>	E-11 / <<enlace>>
OE-0X	<<Valor y escenario>>	<<Conclusión sustentada>>	<<Acción viable>>	E-11 / <<enlace>>
OE-0X	<<Valor y escenario>>	<<Conclusión sustentada>>	<<Acción viable>>	E-11 / <<enlace>>
OE-0X	<<Valor y escenario>>	<<Conclusión sustentada>>	<<Acción viable>>	E-11 / <<enlace>>

7.2 Conclusiones técnicas

Redacte al menos cuatro conclusiones. Cada una debe citar métricas, escenarios, visualizaciones o evidencias; evite repetir actividades realizadas.

7.3 Recomendaciones

Formule al menos tres recomendaciones viables para mejorar consultas, modelo de datos, procesamiento, rendimiento, escalabilidad, calidad, seguridad o visualización. Indique prioridad e impacto esperado.

CAPÍTULO VIII: RESPONSABILIDAD ÉTICA Y PROFESIONAL

8.1 Tratamiento ético, legal y seguro de los datos

Analice privacidad, consentimiento o autorización, propiedad/licencia, seguridad, minimización, anonimización, retención, sesgos, explicabilidad y consecuencias del uso de resultados.

8.2 Impactos de las soluciones

Evalúe impactos organizacionales, sociales, económicos y ambientales de la solución BI y Big Data. Incluya riesgos de decisiones automatizadas o interpretaciones incorrectas.

8.3 Compromiso profesional

Declare responsabilidades del equipo, límites de competencia, transparencia, reproducibilidad y acciones para evitar manipulación o uso indebido de datos y resultados.

REFERENCIAS BIBLIOGRÁFICAS

Incluya todas las fuentes citadas con formato APA vigente. Priorice artículos, libros, documentación oficial, normas y documentación técnica de versiones utilizadas. Verifique que cada cita del texto tenga su referencia y viceversa.

ABET. (2026). Criteria for Accrediting Engineering Programs, 2026-2027.

<https://www.abet.org/accreditation/accreditation-criteria/criteria-for-accrediting-engineering-programs-2026-2027/>

ABET. (s. f.). Assessment Planning.

<https://www.abet.org/accreditation/get-accredited/assessment-planning/>

<<Agregar referencias del proyecto>>

ANEXOS Y MANIFIESTO DE EVIDENCIAS

Complete el manifiesto. Cada evidencia debe existir, abrirse y corresponder con la versión final evaluada. Los archivos extensos pueden alojarse en el repositorio; conserve en el informe su código, descripción y enlace permanente.

CÓDIGO	EVIDENCIA	UBICACIÓN / ENLACE	VERSIÓN / FECHA	RESPONSABLE
E-01	Dataset original y ficha de procedencia	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-02	Dataset procesado y perfil de calidad	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-03	Datos de prueba/sintéticos y método	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-04	Diseño experimental y configuraciones	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-05	Diagrama de arquitectura y esquema de datos	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-06	SQL/NoSQL/Spark, scripts y notebook	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-07	Resultados de la solución BI	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-08	Resultados de la solución Big Data	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-09	Comparación de escenarios y métricas	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-10	Logs, capturas y visualizaciones	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-11	Conclusiones, recomendaciones y decisiones	<<Sección/URL>>	<<Versión>>	<<Nombre>>
E-12	Retroalimentación y mejoras implementadas	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-13	Solución BI funcional / demostración	<<URL>>	<<Versión>>	<<Nombre>>
E-14	Solución Big Data funcional / demostración	<<URL>>	<<Versión>>	<<Nombre>>
E-15	Aportes individuales y defensa	<<Ruta/URL>>	<<Versión>>	<<Nombre>>
E-16	Declaración de uso responsable de IA	<<Anexo/URL>>	<<Versión>>	<<Nombre>>

Lista de anexos sugeridos

1. Anexo A. Requisitos, historias de usuario y criterios de aceptación.
2. Anexo B. Diccionario de datos, modelos y diagramas de arquitectura.
3. Anexo C. Fichas de datasets original, procesado y de prueba.
4. Anexo D. Scripts SQL/NoSQL/Spark, notebooks y archivos de configuración.
5. Anexo E. Logs, tablas completas, visualizaciones y capturas de ejecución.
6. Anexo F. Manual de instalación, despliegue, operación y demostración.
7. Anexo G. Evidencia de retroalimentación, mejoras y aceptación.
8. Anexo H. Historial de repositorio y aportes individuales.
9. Anexo I. Declaración de uso responsable de IA.

DECLARACIÓN DE USO RESPONSABLE DE INTELIGENCIA ARTIFICIAL

La IA puede utilizarse como apoyo, pero no reemplaza el análisis, la simulación, la interpretación de resultados ni la responsabilidad académica del estudiante/equipo.

Responda de manera honesta:

Aspecto	Respuesta
1. ¿Utilicé IA?	☐ No ☐ Sí
2. Herramienta	☐ ChatGPT ☐ Gemini ☐ Copilot ☐ Claude ☐ Otra: _____
3. ¿Para qué la utilicé?	☐ Ideas ☐ Redacción ☐ Código ☐ Errores ☐ Gráficos ☐ Análisis ☐ Otra: _____
4. ¿En qué parte?	☐ Diagrama ☐ Código ☐ Resultados ☐ Análisis ☐ Conclusiones ☐ Reporte ☐ Otra: _____
5. ¿Cómo validé lo que la IA me proporcionó?	☐ Ejecuté el código ☐ Comparé resultados ☐ Revisé fórmulas/unidades ☐ Validé con simulador ☐ Corregí errores

Declaración ética

Confirmando que revisé el contenido final, que ***apliqué pensamiento crítico frente a los resultados proporcionados por la IA***, no ingresé datos personales o sensibles en herramientas externas de IA y asumo la responsabilidad por el trabajo presentado.

Nombre(s)	Firmas
Fecha (dd/mm/yyyy)	

Revisión rápida del docente:

- ☐ Declara uso de IA.
- ☐ Evidencia validación manual.
- ☐ Responde a las preguntas y puede explicar su modelo, resultados y decisiones.

Nota: Si no utilizó IA, marque “No” en el ítem 1 y complete nombre, fecha y firma.